JOHANNES KEPLER
UNIVERSITY LINZ

# The RTL-to-GDSII Flow with LibreLane



Simon Dorrer

**KV VLSI Design, 336.051, WiSe 26/27**

INSTITUTE FOR
INTEGRATED CIRCUITS &
QUANTUM COMPUTING

**JOHANNES KEPLER**
**UNIVERSITÄT LINZ**
Altenberger Straße 69
4040 Linz, Österreich
jku.at

# Today's Goals

After this lecture, you can

- run **LibreLane** and find every result in its run folder
- plan a **floorplan**: core, rows, power grid, pins
- explain **placement**, **buffering**, **clock tree**, **routing**
- read the **signoff**: corners, IR drop, DRC, LVS, LEC
- inspect a run in the **OpenROAD GUI** and read its **reports**
- explain why estimates before routing are **optimistic**
- simulate the **final netlist** with the delays of the layout
- describe how a macro becomes a **chip** with a padframe

Lecture notes: Chapter 10 of iic-jku.github.io/vlsi-design

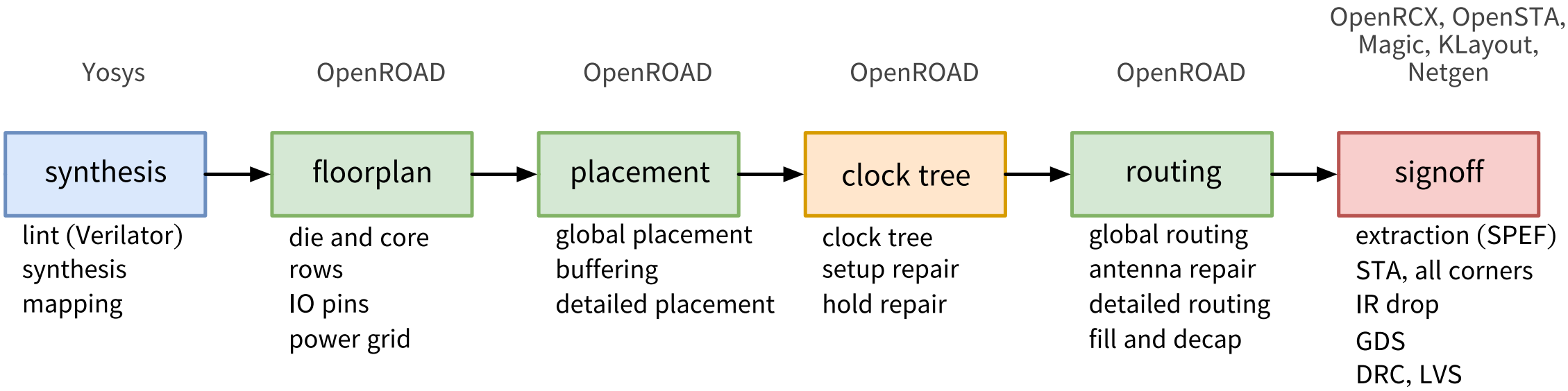


Chapter 10

# LibreLane



# From RTL to Layout




- Each **step** runs one tool on the state the last one left, **checkers** stop on errors
- **Classic** flow hardens a macro, **Chip** flow adds the padframe
- `runs/base/NN-step/`: logs, reports, and the exact `COMMANDS`
- `final/`: GDS, LEF, netlists, SPEF, SDF, metrics

# Configuring a Run

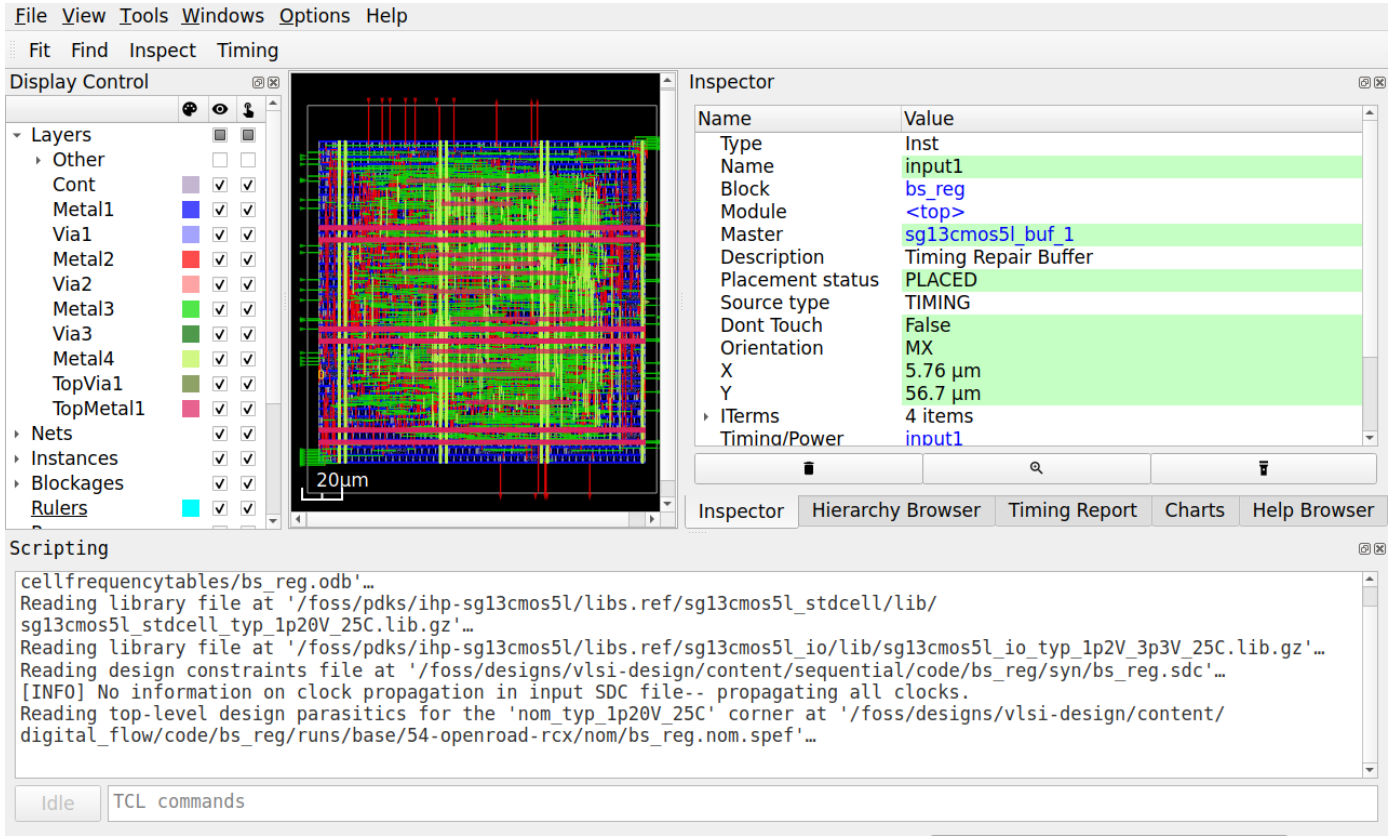
```
DESIGN_NAME: bs_reg
VERILOG_FILES: [.../barrel_shift.sv, .../bs_reg.sv]
CLOCK_PORT: clk_i
CLOCK_PERIOD: 4.0
PNR_SDC_FILE: dir::.../bs_reg/syn/bs_reg.sdc
SIGNOFF_SDC_FILE: dir::.../bs_reg/syn/bs_reg.sdc
FP_SIZING: relative
FP_CORE_UTIL: 50
PL_TARGET_DENSITY_PCT: 60
VDD_NETS: [VDD]
GND_NETS: [VSS]
STD_CELL_LIBRARY: sg13cmos5l_stdcell   # corners from the PDK
```

```
librelane config.yaml --pdk ihp-sg13cmos5l --pdk-root $PDK_ROOT --manual-pdk --run-tag base
```

- Everything not set comes from LibreLane and from `libs.tech/librelane/` of the PDK

# The OpenROAD GUI

- `OpenInOpenROAD`: a flow of one step that opens a run
- Final layout, plus Liberty, SDC, SPEF of the **default corner**
- `-c DEFAULT_CORNER=...` for another corner
- **Display Control**, **Inspector**, Timing Report, Charts, Scripting
- Writes into the run: a step folder, `final/`, `resolved.json`

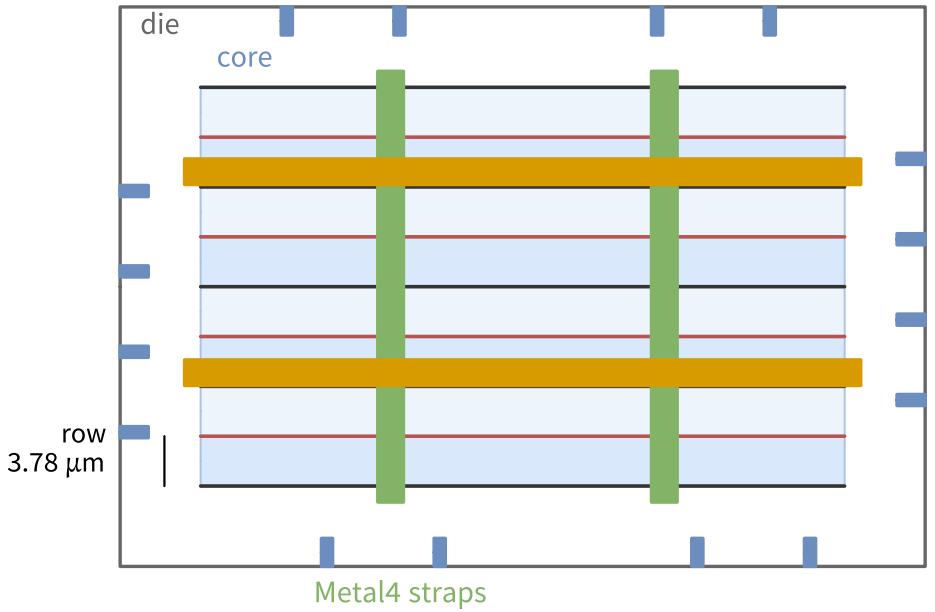



```
librelane config.yaml --pdk ihp-sg13cmos5l --pdk-root $PDK_ROOT \
  --manual-pdk --run-tag base --flow OpenInOpenROAD
```

# Floorplan



# Die, Core, Rows, and Power




Metal1 rails: VDD (red), VSS (black)

TopMetal1 straps

pins on Metal3 (left, right),
Metal2 (top, bottom)

- `FP_CORE_UTIL`: core = cell area / utilization
- Rows of **3.78 µm**, sites of **0.48 µm**
- No tap cells: the cells tie their wells
- Metal1 rails, straps on **Metal4** and **TopMetal1**, 50 µm pitch
- Pins on Metal2 and Metal3

**Exercise** (Section 10.3): plan the floorplan, find the straps in the GUI, make the core denser.

# Placement



# Placement and Buffering

- **Global placement**: cells repel like charges, nets pull, target density
- STA **estimates** each wire: tree length × R and C per micron
- **Repair of the design**: slew, capacitance, fanout above **8**
  - a `buf_1` at each of the 40 inputs and 32 outputs
  - 142 more buffers split 30 high-fanout nets into trees
- **Detailed placement**: every cell onto a site of a row

# Clock Tree Synthesis

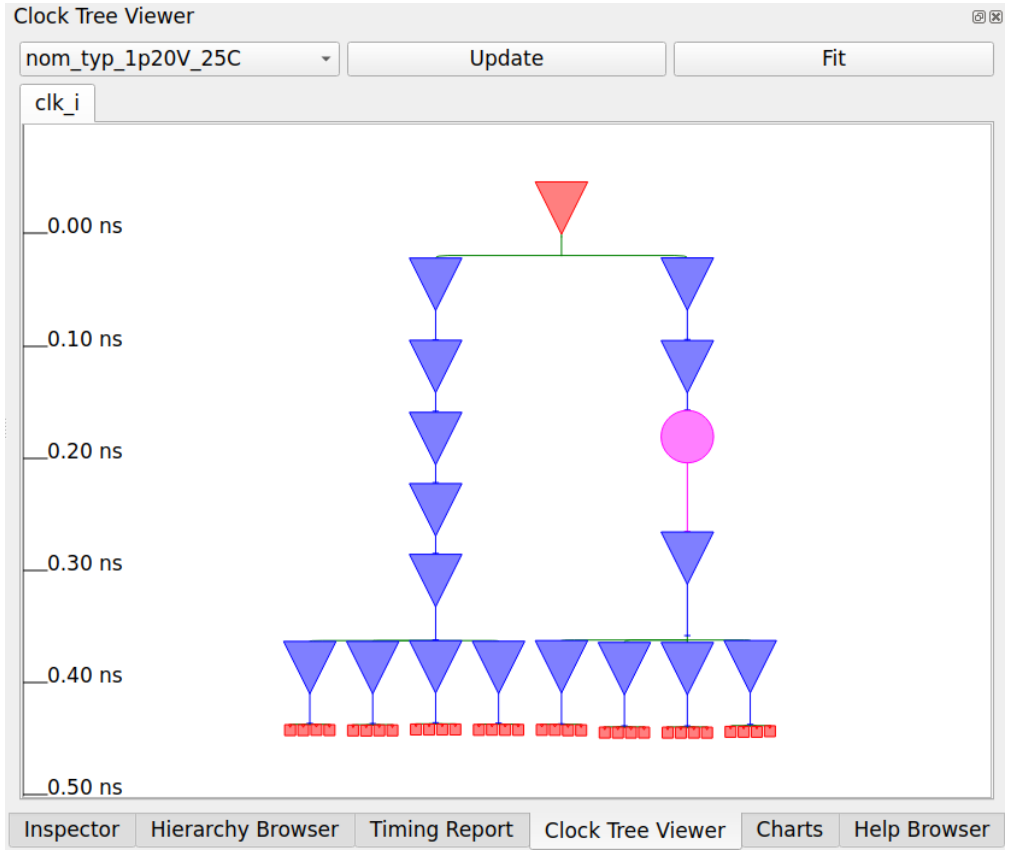


# Clock Tree Synthesis

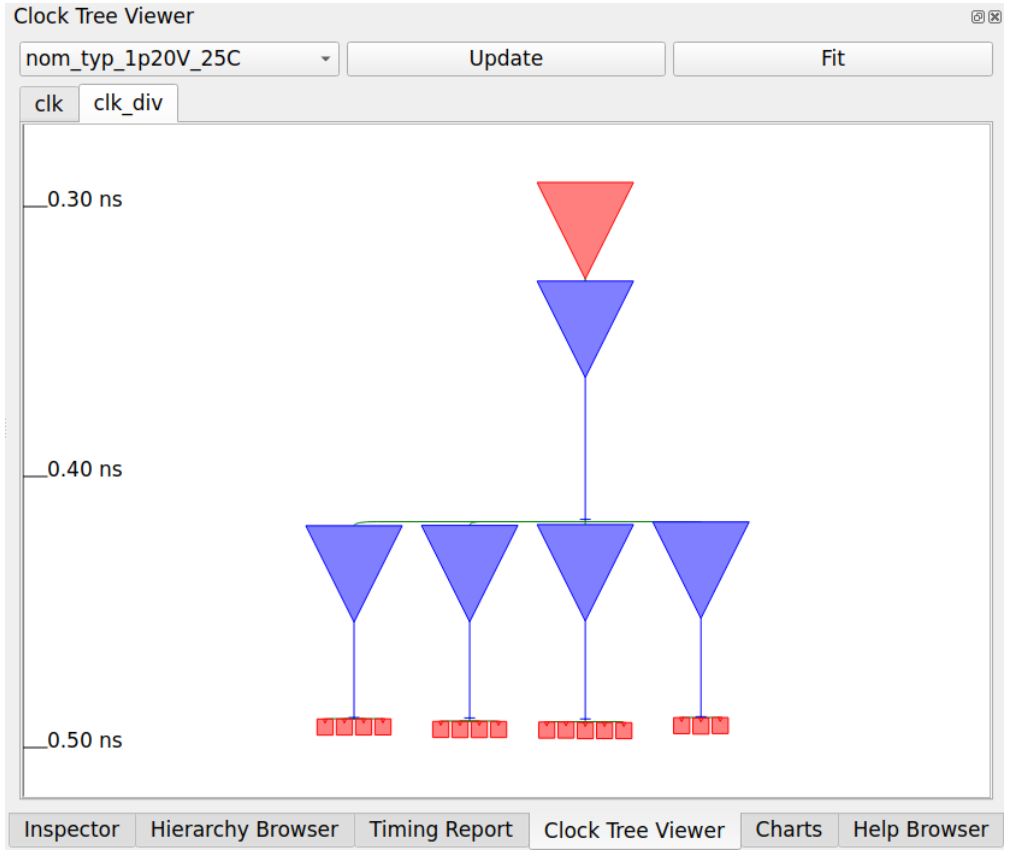
- Until now an ideal clock, now a **tree**: equal **latency**, small **skew**
- TritonCTS: **H-tree** from the PDK clock buffers (`buf_8` ... `buf_1`)
- STA from now on: the **propagated** clock
- Timing repair: larger cells for setup, `dlygate` delay cells for hold
- Clock gate (`cg_mix`): CTS **balances** it with 3 delay buffers
- Divided clock (`div_mix`): two trees, **no balancing**, 32 delay cells for hold

**Exercise** (Section 10.5): follow the clock and the reset, ask for more hold margin.

# The Clock Tree Viewer




`cg_mix`: clock gate, three delay buffers




`div_mix`, `clk_div`: behind the divider

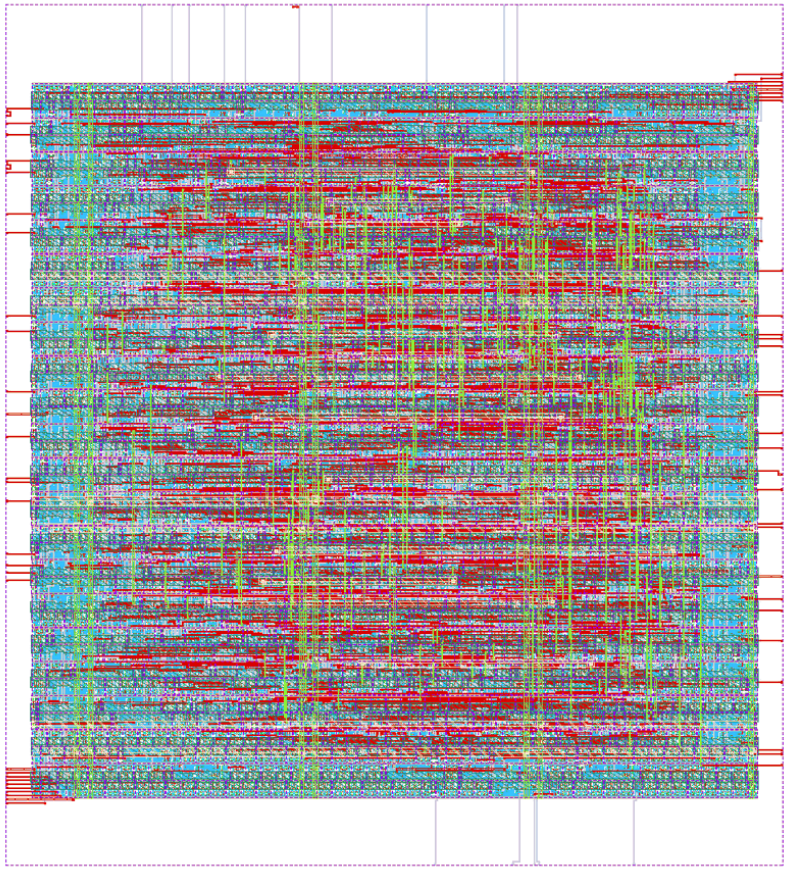
Time runs downward: a balanced tree ends in one row of flip-flops

# Routing



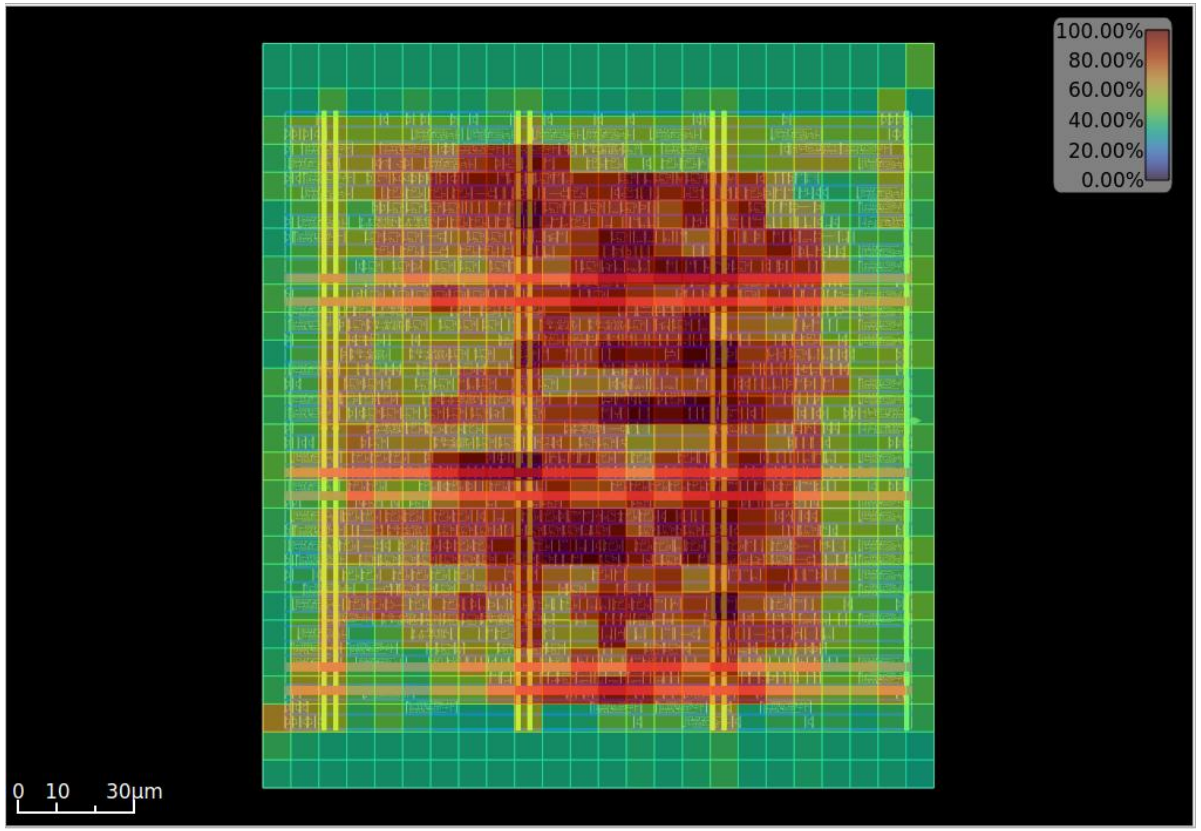
# Routing and Fill

- **Global routing**: tiles, layers, congestion
  - 44 % of Metal2, 39 % of Metal3, no overflow
- **Antenna rule**: jumper or diode `antennanp`
- **Detailed routing**: repairs violations in iterations until none are left
- **Fill**: decaps and fillers in the gaps



# Routing Congestion in the GUI

- `Heat Maps`, `Routing Congestion`: the tiles of the **global router**
- Open the state **after global routing**
  - `--with-initial-state`, under a run tag of its own
- `base`: middle **82 %**, margins **38 %**
- 10 full tiles, but none overflows
- Also: placement density, RUDY, power, pins, IR drop

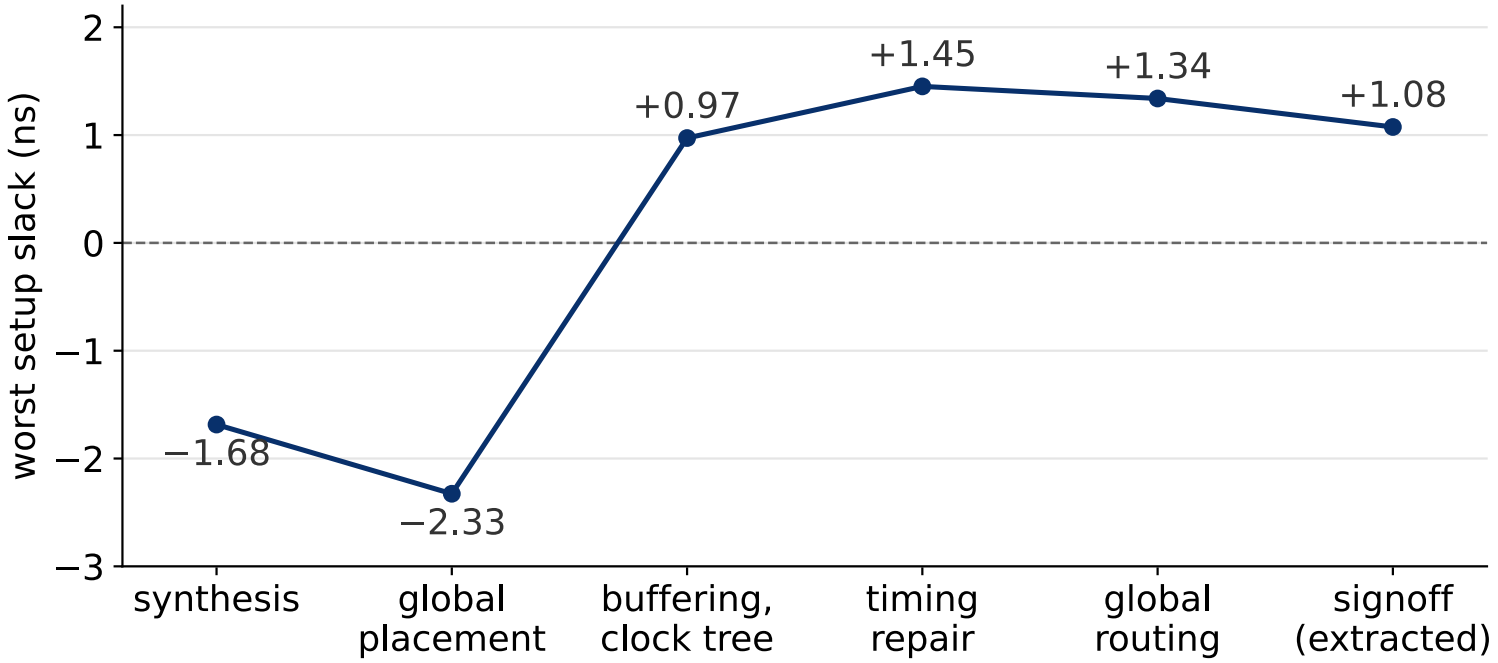



**Exercise** (Section 10.6): find the congestion of the denser core, and compare its final state.

# Signoff



# Extraction and Timing




- **OpenRCX** extracts R and C of every wire (SPEF), STA checks **all corners**
- Estimates before routing are **optimistic**: detours, vias, coupling

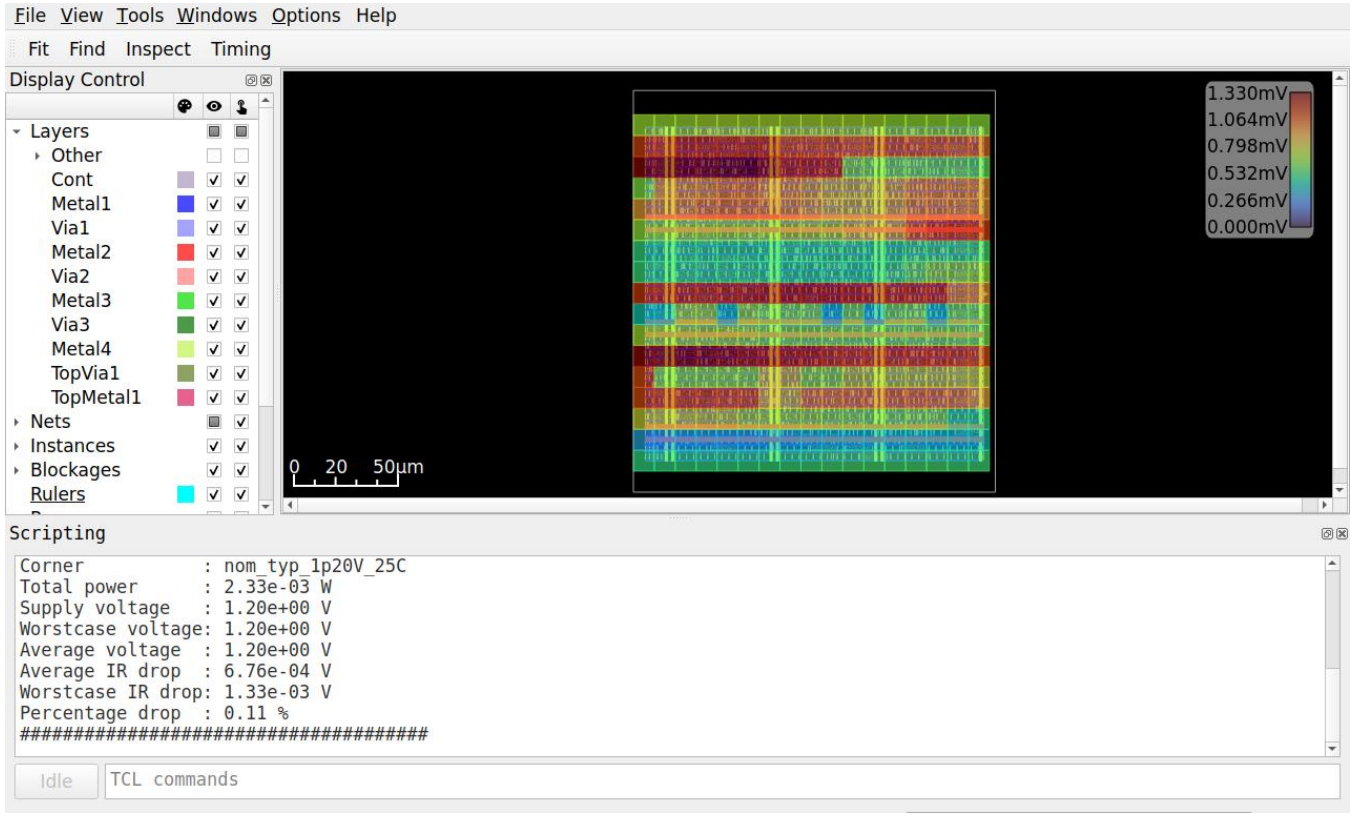
# Checking the Layout

- **IR drop** in the power grid: 1.33 mV worst, 0.11 % of 1.2 V
- **DRC** with Magic and KLayout, and an XOR of the GDS files from Magic and KLayout
- **LVS**: Magic extracts, Netgen compares, cells as abstracts
- **LEC** with EQY, run by hand: the final netlist against the RTL

**Exercise** (Section 10.7): find what fails in signoff, close the timing, prove the netlist.

# Timing and IR Drop in the GUI

- **Timing Report**: worst paths, drawn into the layout
- **Charts**: histogram of the endpoint slack
- Both: default corner only
- `analyze_power_grid -net VDD`, then `Heat Maps`, `IR Drop`
- VDD: **1.33 mV** in the top rows, between two straps



# The Reports of a Run

- `Flow complete.` is **not necessarily** a clean design
  - the setup checker counts only the typical corners, so a run can complete with a failing corner
  - `manufacturability.rpt`: antenna, LVS, DRC, no timing
- Timing: `summary.rpt`, per corner `max.rpt`, `violator_list.rpt`
- Limits, power, IR drop: `checks.rpt`, `power.rpt`, `irdrop.rpt`
- `warning.log`, `config.json` of each step, and `final/metrics.json`

**Exercise** (Section 10.7.1): read the IR drop of `VSS`, the warnings, and the power of `bs_reg`.

# Post-Layout Simulation



# Post-Layout Simulation

- Same testbench on `final/nl/`, one SDF per **corner** from `final/sdf/`
- **Wires** as `INTERCONNECT`, annotated only with `-ginterconnect`
- **Clock tree**: every flip-flop sees `clk_i` late
- Testbench as in the **SDC**: inputs 1.0 ns after, outputs 1.0 ns before each edge
- **Prove** the annotation: one path in the waveform = its SDF delays
- `-Tmin`, `-Ttyp`, `-Tmax` pick one value of each triple

```
iverilog -g2012 -gspecify -ginterconnect -o pl.vvp \
  $CELLS/sg13cmos5l_udp.v $CELLS/sg13cmos5l_stdcell.v \
  runs/base/final/nl/bs_reg.nl.v tb/tb_bs_reg.sv
C=nom_typ_1p20V_25C
vvp -n pl.vvp +SDF=runs/base/final/sdf/$C/bs_reg__$C.sdf
```

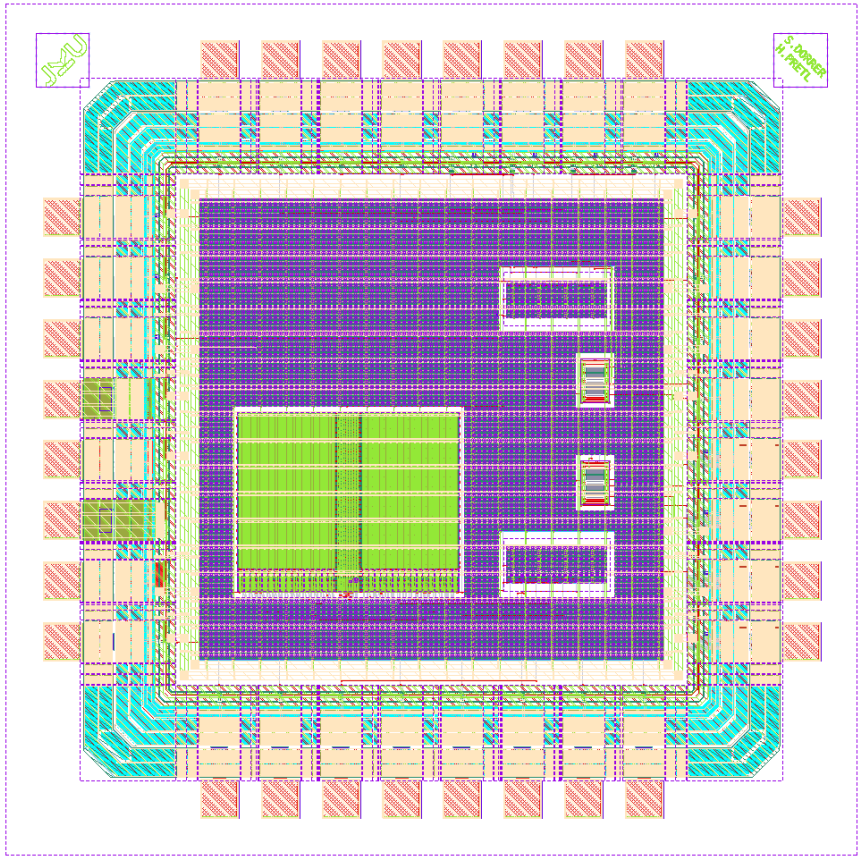
**Exercise** (Section 10.8): follow a clock path, drive the inputs early, try the slow corner.

# Chip Assembly



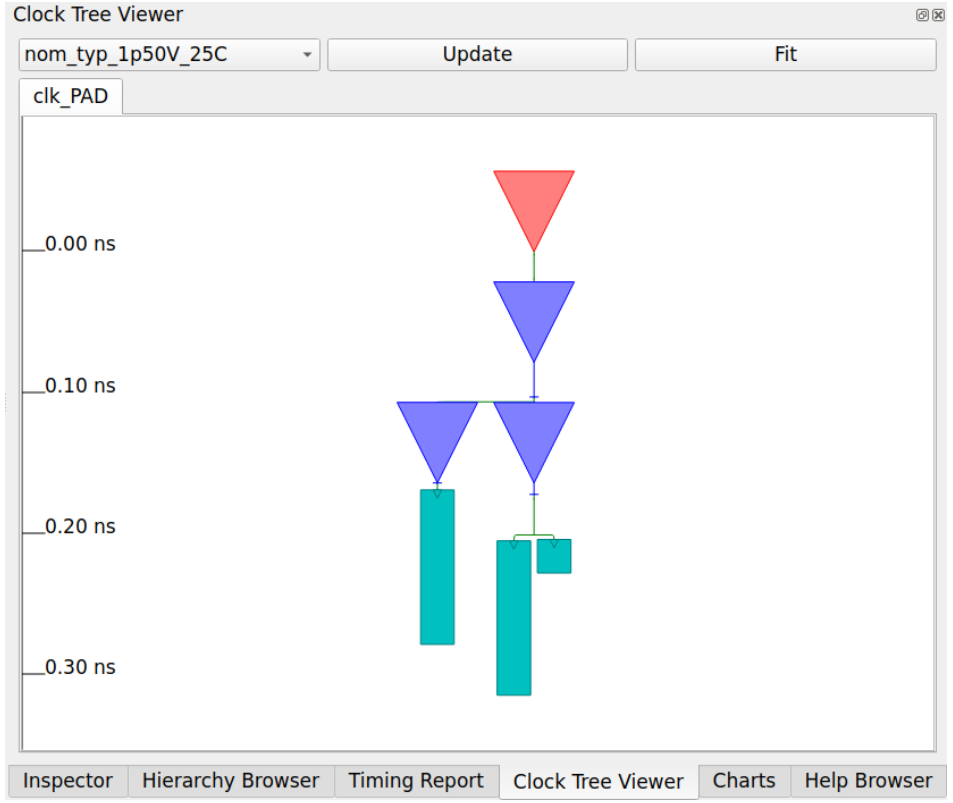
# From Macro to Chip

- AMS chip template: LibreLane's **Chip** flow
- **Padframe**: IO cells with bondpads, ESD, supplies
  - 1.6 × 1.6 mm, 32 pads in four lists, `PAD_WEST` to `PAD_EAST`
- **Seal ring** along the edge of the die
- `MACROS`: GDS, LEF, netlists, timing per macro
- Core ring to the supply pads, non-default rules for analog nets
- Then chip logo, fill, DRC, bonding diagram



# The Chip in the GUI

- `make librelane-openroad`: the last run in the GUI
- `make copy-reports`: copies the reports
- Chip clock: three buffers, two counters, one SRAM
- A counter is a **bar**: its internal clock latency
- Hardened macro: **no power** in its model, so it is signed off in its own run



# Summary

- LibreLane: **steps** with their own folders, results in `final/`
- Floorplan: core from **utilization**, rows, Metal1 rails and straps
- Placement buffers every net beyond **slew, cap, fanout** limits
- CTS: **H-tree**, propagated clock, setup and hold repair
- Routing: global, antenna, detailed, then **fill**
- Signoff: **extraction**, all corners, IR drop, DRC, LVS, then **LEC**
- GUI: clock tree, congestion, paths, IR drop, default corner only
- `Flow complete.` is not clean: read the **reports**
- Post-layout simulation: SDF per corner, drive as the **SDC** says
- Chip: macros in a core inside a **padframe** and a seal ring

# Exercises

1. Plan the Floorplan (10.3)
2. Follow the Clock and the Reset (10.5)
3. Find the Congestion (10.6)
4. Close the Timing (10.7)
5. Read the Reports (10.7.1)
6. Simulate the Layout (10.8)



Chapter 10

All code in `content/digital_flow/code` of the course repository, every number reproduced by `scripts/check_digital_flow.sh`



JOHANNES KEPLER
UNIVERSITY LINZ